Post-Silicon Deception: Evasive Hardware Trojan Through Adversarial Power Trace

Behnam Omidi, Ihsen Alouani, *Senior Member, IEEE*, and Khaled N. Khasawneh, *Member, IEEE*

***Abstract*—The globalization of the Integrated Circuit (IC) supply chain, driven by time-to-market and cost considerations, has made ICs vulnerable to hardware Trojans (HTs). Against this threat, a promising approach is to use Machine Learning (ML)-based side-channel analysis, which has the advantage of being a non-intrusive method, along with efficiently detecting HTs under golden chip-free settings. In this paper, we question the trustworthiness of ML-based HT detection via side-channel analysis. We introduce an HT obfuscation (HTO) approach to allow HTs to bypass this detection method. Rather than theoretically misleading the model by simulated adversarial traces, a key aspect of our approach is the design and implementation of adversarial noise as part of the circuitry, alongside the HT. We detail HTO methodologies for ASICs and FPGAs, and evaluate our approach using the TrustHub benchmark. Interestingly, we found that HTO can be implemented with one single transistor for ASIC designs to generate adversarial power traces that can fool the defense with 100% efficiency. We also efficiently implemented our approach on a Spartan 6 Xilinx FPGA using 2 different variants: (i) DSP slices-based, and (ii) ring-oscillator-based design. Additionally, we assess the efficiency of countermeasures like spectral domain analysis, and we show that an adaptive attacker can still design evasive HTOs by constraining the design with a spectral noise budget. In addition, while adversarial training (AT) offers higher protection against evasive HTs, AT models suffer a considerable utility loss, potentially rendering them unsuitable for such a security application. We believe that this research represents a significant step in understanding and exploiting ML vulnerabilities in a hardware security context, and we make all resources and designs openly available online.[1]**

***Index Terms*—Adversarial attack, hardware trojans, machine learning.**

I. Introduction

THE increasing demand for shorter time-to-market and cost-effective hardware design and manufacturing has globalized the IC supply chain, exposing ICs to hardware-based security threats like counterfeiting, IP piracy, reverse engineering, and HTs due to outsourcing to untrusted entities. The modern IC production life-cycle can be divided into three main phases: *design, fabrication, testing* [1]. HTs are malicious modifications of a circuit to control, modify, disable, monitor, or affect the operation of the circuit. Fig. 1 illustrates the standard SoC design life cycle, from system-level specification to fabrication. Various stages in the IC supply chain are potential points for hardware Trojan (HT) injection. This could be a malicious in-house designer or team that can alter the hardware design at different abstraction levels while relying on third-party IPs or outsourcing manufacturing facilities could create a vulnerability for adversaries to tamper with and infect the design.

Underestimating HT threats can lead to severe consequences. For example, F-Secure's 2020 report on counterfeit Cisco switches highlighted the risks of HTs in bypassing authentication processes [2]. There have been other instances, like Bloomberg's claims in 2018 and 2021 about compromised servers in data centers of major companies, although these were disputed [3]. Additionally, the 2008 bombing of a Syrian nuclear facility after a backdoor disabled its radar potentially due to an HT [4]. These threats pose significant risks to various critical systems, making the integrity of the semiconductor supply chain a pressing concern globally.

Various methods have been proposed to detect HTs in chip design and manufacturing, which include destructive and non-destructive techniques [5]. Destructive techniques involve dissecting chips to reverse engineer their layers [6]. Although these methods can be highly accurate, they are costly, time-consuming, and render the chip unusable, limiting their practicality to a single chip analysis. In contrast, non-destructive methods focus on the chip's functionality or side-channel signals, offering more practical alternatives [7]. These include Logic Testing [8], IP Trust Verification [9], Optical Detection [10], Side-Channel Analysis [11], and ML-based detection [12]. In particular, an ML-based approach for HT detection using power side-channel traces is an interesting post-silicon approach because it does not require a gold chip, beyond being highly efficient and non-intrusive [13].

In this paper, we question the trustworthiness of ML-based HT detection using power side-channel data.

Research challenges: While the vulnerability of ML models to adversarial noise is established in the literature, generating **adversarial circuits**, i.e., adversarial noise implementable in circuitry, is a novel problem. In fact, the model's input in power-based HT detectors is a power trace and the adversarial noise generated by existing methods should mimic the behavior of noise in hardware. Therefore, our first challenge is **(C1)**:

Received 4 October 2024; revised 11 March 2026; accepted 8 May 2026. Date of publication 25 May 2026; date of current version 1 September 2026. This work was supported by National Science Foundation under Grant CCF-2212427 and Grant CNS-2155002. *(Corresponding author: Behnam Omidi.)*

Behnam Omidi and Khaled N. Khasawneh are with ECE, George Mason University, Fairfax, VA 22030 USA (e-mail: bomidi@gmu.edu; kkhasawn@gmu.edu).

Ihsen Alouani is with CSIT, Queen's University Belfast Northern Ireland, BT71NN Belfast, U.K. (e-mail: i.alouani@qub.ac.uk).

This article has supplementary downloadable material available at https://doi.org/10.1109/TDSC.2026.3696769, provided by the authors.

Digital Object Identifier 10.1109/TDSC.2026.3696769

[1]https://github.com/beomidi/HW-Trojan-UAP-Generation.

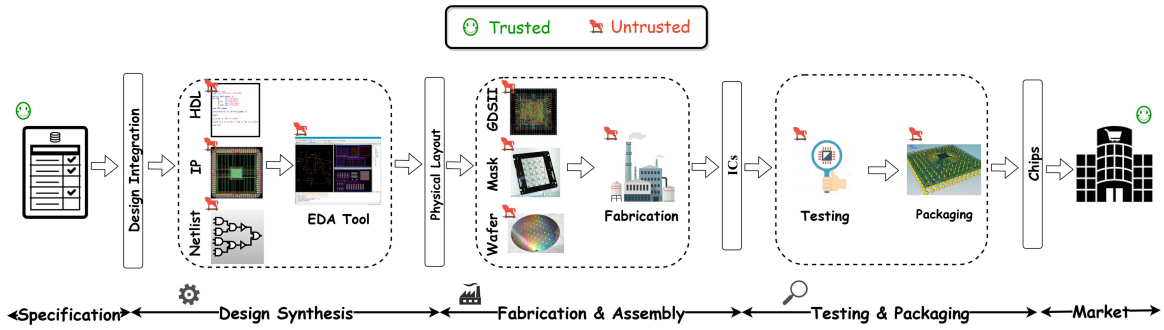


Fig. 1. The credibility level of IC production life-cycle.

given an adversarial power trace, *how to generate an adversarial circuit that consumes this trace?* Moreover, given the threat model of HT detection, the second challenge **(C2)** is that *adversarial circuits* need to be automatically designed with the lowest possible resource utilization. Finally, the third challenge **(C3)** is how to design defenses and to what extent these defenses can be bypassed by the attack.

Attack: We propose HTO, a method that exploits the vulnerability of ML to adversarial noise to generate evasive HTs that are undetectable by ML-based approaches. We first generate a targeted adversarial patch that takes hardware noise into account and successfully fools a state-of-the-art HT defense approach. We then propose new approaches for designing obfuscation circuits that consume adversarial power traces for both ASICs and FPGAs. To make the attack even stealthier, we optimize the generated adversarial noise to minimize the required circuitry to obfuscate the HT.

We evaluated HTO using the TrustHub benchmark with a recent ML-based HT detection technique and showed that our approach efficiently bypasses the defense. We also demonstrate that HTO can be implemented with **a single transistor** for ASIC designs, making it very stealthy.

Countermeasures and adaptive attack: For a comprehensive security evaluation, we investigate defenses and propose an adaptive attack. We propose a spectral defense in the *frequency domain* to improve the robustness of ML-based detection and investigate an adaptive attack that generates noise in a specific frequency range to bypass this defense. Additionally, we examine AT as a countermeasure against HTO, but find that it can be circumvented with increased attacker power budget and resulting in reduced baseline accuracy.

To the best of our knowledge, this is the first work to practically exploit ML vulnerabilities to design adaptive hardware security attacks.

Contributions: The key contributions of this paper can be summarized as follows:

Evasive HT: We propose and develop an adversarial power generation methodology that allows HTs to bypass ML-based side channel defense. We design configurable adversarial patch generation circuits for various platforms (ASIC and FPGA) to consume **adversarial** power during the activation of HT to bypass the power-analysis detection. (Sections III, IV)

Attack optimization: We optimized the generated noise to achieve minimal resource utilization and showed that an effective HTO design can be implemented with only 1 Transistor in an ASIC. (Section V)

Countermeasures and adaptive attacks: We propose a spectral domain countermeasure for HTO (Section VII) and explore an adaptive attack by considering the spectral domain adversarial noise budget (Section VIII). We also explore the practicality of adversarial training, taking into account the utility constraints of the security use case.

II. Threat Model

In this paper, we consider the adversary's objective to be *evade post-silicon power-based ML detection of the HT.*

Attacker's capabilities: The attacker within this threat model has assumed to be a malicious insider in the pre-silicon design phase—either at the foundry or an IP vendor—with the capability to compromise the validation workflow and modify the design to insert hardware Trojans (HTs) at any point in the victim circuit, as is commonly assumed in state-of-the-art scenarios [13], [14], [15], [16]. They can inject additional circuitry, but cannot alter the defender's ML-based HT detector, which is trained to detect the existence of a triggered HT through power analysis.

Defender: We assume the defender uses ML-based HT detectors, which analyze the circuit power trace as golden chip-free techniques. We trained ML classifiers, including $ResNet-18$, $VGG-11$, SVM, and $HTnet$ [13], based on power signals gathered on TrustHub benchmarks [17] to detect HTs.

Attacker's knowledge: We assume that the attacker is aware of the defender's use of this ML model but sees it as a black-box; unable to modify its parameters or directly interfere with its operation.

Objective: An adversary, using information learnt about the ML-based defense mechanism, trains a proxy model and tries to craft power trace perturbations added to the ML input to force it to output wrong labels.

This paper focuses on evaluating the robustness of machine learning-based detection mechanisms using power side channels. Detection methods based on other mechanisms, such as physical inspection or functional testing, are outside the scope of this work.

III. Adversarial Power Trace Generation

In this section, we introduce the process of generating an adversarial patch, which is a sneaky power trace augmented to

Algorithm 1: Adversarial Power Trace Generation.

Input: Data set: $\mathcal{D}$, classifier: C, noise magnitude: ε, standard deviation: σ, number of iterations: N.
Output: δ Adversarial power trace.
Initialize $\delta \leftarrow rand()$
while $iter < N$ **do**
  for each Batch $X_i \sim \mathcal{D}$ **do**
    for all x_j s.t. $C(x_j) = 1$ **do** ▷ //HT distribution.
      $\delta \leftarrow \frac{1}{|X_i|}\sum_{j=1}^{|X_i|}\{\{\alpha sign(\nabla_x J_\theta(C(x_j+\delta),\ell))\}\}$
      $\delta \leftarrow Clip_\varepsilon\{\delta\} + z \sim N(0,\sigma^2)$
    end for
  end for
  $iter+ = 1$
end while

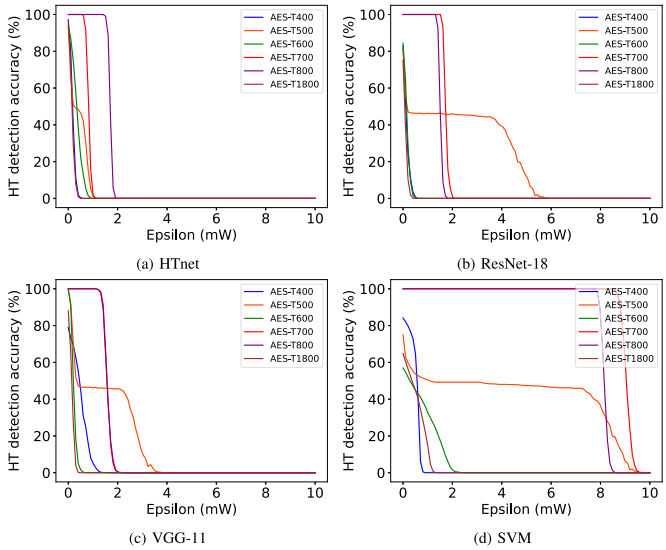


Fig. 2. HT detection accuracy based on changing the maximum power budget.

the circuit power to force the ML-based HT detector model to wrongly label the HT-inserted ICs. We specify that it is an adversarial **patch** because the noise is intended to be "*universal*"—that is, it should fool the model regardless of the specific input sample. This is contrary to classical adversarial noise that is crafted for a given input sample. In the following, we use adversarial patches interchangeably with adversarial noise.

Problem formulation: Given an original input power trace x and a victim classification model $C(.)$, the problem of generating an adversarial example x_{adv} can be formulated as a constrained optimization:

$$C(x+\delta) \neq C(x),\ \forall\, x \sim \mu_{HT}, and$$
$$\|\delta\| = \mathcal{D}(x, x+\delta) \leq \varepsilon \qquad (1)$$

Where μ_{HT} is the data distribution corresponding to triggered hardware Trojans, which is our target for a classifier $C(.)$ to be fooled, i.e., evading detection, and $\mathcal{D}$ is the difference between the HTO and the original HT power. This means that we want to generate one noise (δ) that has a magnitude within the range of ε.

Distance Metrics: For evading detection, the adversarial perturbations should be kept minimal. To measure the magnitude of adversarial noise, L_p metrics are generally used in the literature [18], [19], [20]. In our case, the samples are power traces, and L_∞ is more appropriate to measure the detectability of a potential power trace, since L_∞ measures the maximum difference for all elements at the corresponding positions in the two samples.

Adversarial Patch Generation: We generate the adversarial power noise by solving the optimization problem and considering the detection loss function. Given a batch of power traces X_i and the corresponding target labels ℓ. In each iteration, the algorithm adds the precalculated perturbation to the Trojan inserted trace and adjusts the new noise along the direction of its gradient w.r.t. the inference loss $\partial J/\partial x$. Algorithm 1 describes the generation of the adversarial power noise process.

Where the attack strength is determined by α, which is the perturbation constraint, $C(x_j+\delta)$ computes the output of the ML-based classifier, J_θ is the cross entropy loss function, and the sign function is indicated as $sgn(\cdot)$. To produce a universal patch for the traces in the dataset, the average of all generated noise has been forwarded to the next iteration. Since the negative values of the patch do not have any physical meaning in terms of power consumption and to ensure that the perturbation is restricted to the specified power budget ϵ, the attack is followed by a clipping operation ($0 \leq \delta \leq \epsilon$). A Gaussian noise z with the mean value of 0 and standard deviation σ has been added to the perturbation to add robustness to the measurement and environment noise.

To evaluate the correctness of the algorithm, we generate patches for power signals gathered on the TrustHub benchmark based on different power budgets (ϵ) for $ResNet-18$, $VGG-11$, SVM, and $HTnet$ models. The effectiveness of the generated patches has been illustrated in Fig. 2, which clearly indicates that we can fool all models with 100% efficiency by increasing the noise magnitude.

Moreover, it is important to note that the only publicly available dataset suitable for training ML-based HT detections using power side-channel signals is that of [13], which focuses exclusively on an AES circuit. To demonstrate the generality of our approach beyond this specific use case, we generated new datasets based on a variety of benchmarks available on TrustHub [21]. The results obtained from these new datasets, presented in Appendix Figure S1 and Table S1, available online, clearly indicate that the proposed method is not limited to the AES module and generalizes effectively to other circuit designs.

IV. HTO: Proposed Adversarial Circuit Design

In the previous section, we generated an adversarial patch that is able to fool a power-based HT detection system. However, the adversarial noise we consider in Section III is a simulated additive noise and, in a practical scenario, needs to be produced directly in the circuit. Therefore, in this section, we propose a methodology to design circuitry that consumes a given power trace, which is the adversarial power trace generated in

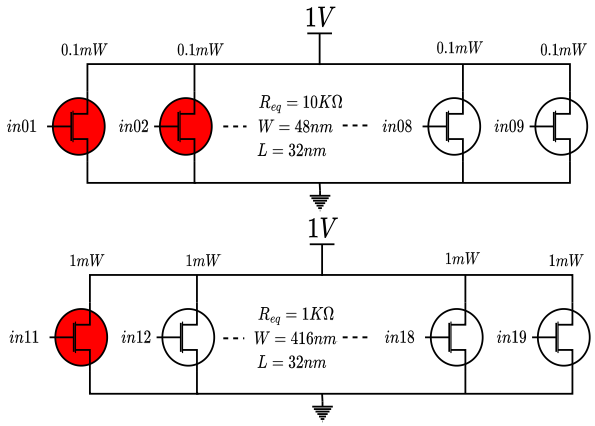


Fig. 3. ASIC adversarial power network example.

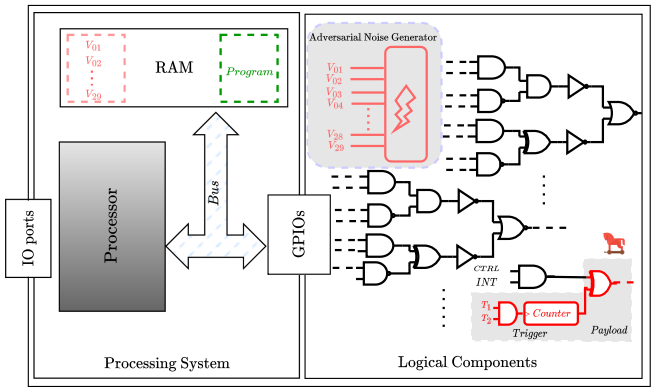


Fig.4. Illustration of a configuration vector generator in a SoC.

Section III. Our objective is to answer the following question: **Given a target (adversarial) power trace, how to design a circuit that consumes a trace emulating the target trace?**

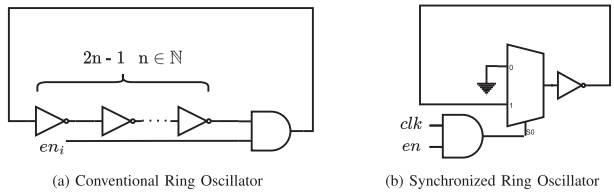
In the following, we propose a design methodology for both FPGA and ASIC platforms. Specifically, we implement a transistor-based circuit for ASICs and both Ring Oscillator (RO)-based and DSP-based designs for FPGAs to emulate the adversarial patch on each platform.

A. HTO Circuit Design for ASICs

For designing a circuit to generate adversarial power noise in an ASIC, we propose a configurable power consumption unit. Specifically, the configurable power consumption unit is a network of intrinsic resistors hidden in transistors that are able to be configured by activating transistors. The hidden resistors can be specified by the feature parameters of transistors during the design phase. Based on the samples in the adversarial power noise generated by Algorithm 1, the equivalent hidden resistors are calculated, and corresponding vectors are activated in the power network. An example of this conversion for $P = 1.2$ mW and $V = 1\,v$ with 18 transistors has been shown in Fig. 3 and Algorithm 2.

There are multiple ways to store configuration vectors:

- Vectors can be stored in a Block-RAM where the order of vectors reflects the consumed power in each cycle by the HTO.



(a) Conventional Ring Oscillator (b) Synchronized Ring Oscillator

Fig. 5. Ring Oscillator.

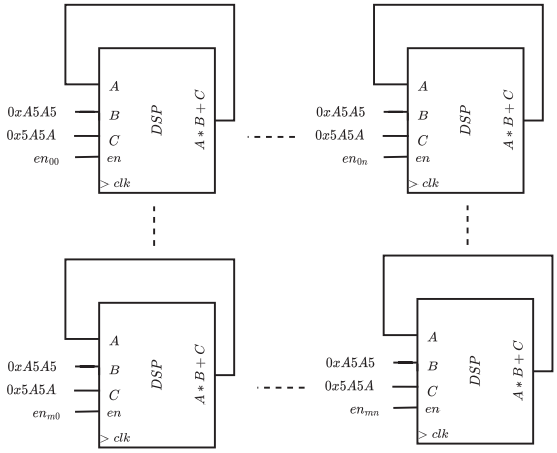


Fig. 6. DSP-based network design.

Algorithm 2: Configuration Vector Generator.

Input: Noise samples: $\mathcal{N}$
Output: Configuration vectors: $\mathcal{V}$
Initialize $\mathcal{V} \leftarrow 0$
for each noise sample P_i in $\mathcal{N}$ **do**
  for each digit d_l in P_i **do**
    for each cell c_j in a raw of circuit **do**
      if $j < d_l$ **then**
        $\mathcal{V}_{lj} = 1$
      else
        $\mathcal{V}_{lj} = 0$
      end if
    end for
  end for
end for

- For a System on Chip (SoC) circuit that has a processor, it is recommended to store the vectors in the processor's RAM to minimize resource utilization. The vectors can feed to the configurable power network through a port connected to the processor's bus. The structure of stored vectors in the SoC system has been shown in Fig.4.

B. HTO Circuit Design for FPGAs

Not having access to the parameters of transistors, we take a different approach to designing an HTO circuit in the FPGA platform. In this context, we require power-dense components that can be easily toggled to induce measurable and controllable variations in the power profile. Any logic element—such as LUTs, flip-flops, or BRAMs—that supports controlled

switching activity can be leveraged to generate such power perturbations.

In our setup, we propose two distinct logical circuits: one based on ring oscillators and the other on DSP networks. These designs are selected for their fine-grain control and predictable power consumption characteristics, making them well-suited for adversarial power injection. The implementation details of each circuit are outlined below:

1) Ring Oscillators: A network of configurable ring oscillators, illustrated in Fig. 5 has been proposed to consume generated adversarial power traces on an FPGA. The ring oscillator frequency of the oscillation formula is:

$$f = \frac{1}{2n\tau} \tag{2}$$

Where τ is the propagation delay for a single inverter, and n is the number of inverters, which should be an odd number. For generating a sneaky adversarial power, the number of inverters should be as minimal as possible (one inverter), leading to the maximum frequency and power consumption. Unfortunately, our experimental setup can not capture the power consumed by the conventional ring oscillator illustrated in Fig. 5(a). Therefore, we modified our ring oscillator design to the one shown in Fig. 5(b) to synchronize our circuit with the whole system and capture the power precisely. In the new design, by logically-anding the enable signal with the FPGA clock, we synchronize the capture power devices with the clock generated by the ring oscillator. We can consume around 1 mW by using two ROs in our experimental setup, limiting our resolution to 1 mW.

2) DSPs: We also exploit DSP components in an FPGA to implement an obfuscated design that evades the hardware Trojan detection technique. We observe that, with the DSP configuration shown in Fig. 6, the design consumes approximately 1 m Wof power per clock cycle in our experimental setup. It should be noted that this is only one possible configuration for our FPGA implementation. Alternative component types or configurations may be leveraged depending on the available spare resources and the characteristics of the target design.

Configuration vectors: Similar to the ASIC design, the configuration vectors can be stored in Block-RAMs, or for an SoC circuit, vectors can be stored in the processor's RAM and sent to a general I/O port connected to the HTO circuit.

C. Experimental Setup

Different experimental setups have been used for the ASIC and FPGA platforms to generate the adversarial power traces. For capturing the power in an FPGA, we leverage ChipWhisperer 305 with Xilinx Spartan 6 as a target device and ChipWhisperer-lite as a capture device. As shown in Fig. 7, the control of Artix-7 and Spartan-6 on the FPGA is managed by a USB connection from the laptop. Control information of our power measurements is sent through the 20-pin connector connected to ChipWhisperer-lite. The power measurements will be done through an SMA cable to the ChipWhisperer-lite and sent to the laptop through a USB connection. In our setup, we synthesized a ring oscillator and a DSB-based network on the target FPGA. During each clock cycle, power consumption was captured using the ChipWhisperer 305. The collected power traces were then analyzed to assess their influence on the dataset and compared against those generated by the algorithm.

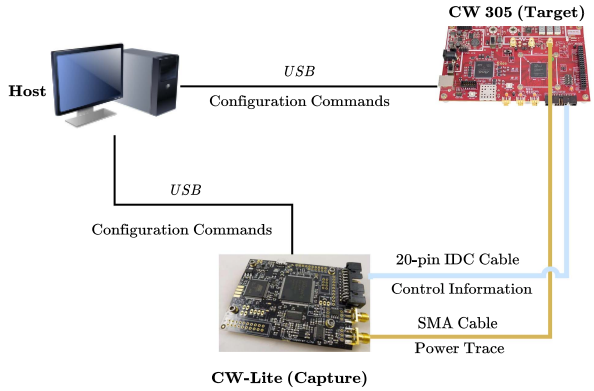


Fig. 7. FPGA setup.

On the other hand, in ASIC implementation, adversarial patches designed to fool the classifier were simulated using $HSpice$ 2019 with the 32 nm low-power Predictive Technology Model (PTM) library. A $Python$ script was developed to automate the generation of equivalent transistor-level circuits that replicate the adversarial samples, using appropriate configuration vectors in HSpice format. Power traces for each evasive Trojan-equivalent circuit were collected from HSpice simulations. These traces were then analyzed to evaluate their impact on the dataset and compared against those generated by the algorithm.

D. Results

Tables I and II illustrate HT detection accuracy, resource utilization, and required noise budget to drop accuracy below 50% on ASIC and FPGA, respectively. The results for ASIC illustrate the negligible difference between the generated noises and simulated ones with the mean square error of 0.0002. The power consumption resolution on the FPGA restricts us to only implementing the patches with the granularity of 1 mW. Hence, the accuracy of the simulation drops in a way that the value of 0.31 is reported for the mean square error.

Discussion: Our results show that the generated HTO consumes power traces that accurately emulate the adversarial patch impact on the victim system. However, the high resource utilization required by the HTO is a clear limitation of its threat, which can be easily exposed by state-of-the-art techniques. In the following, we will investigate optimization techniques to reduce resource utilization for more sneaky HTO designs.

V. HTO Optimization

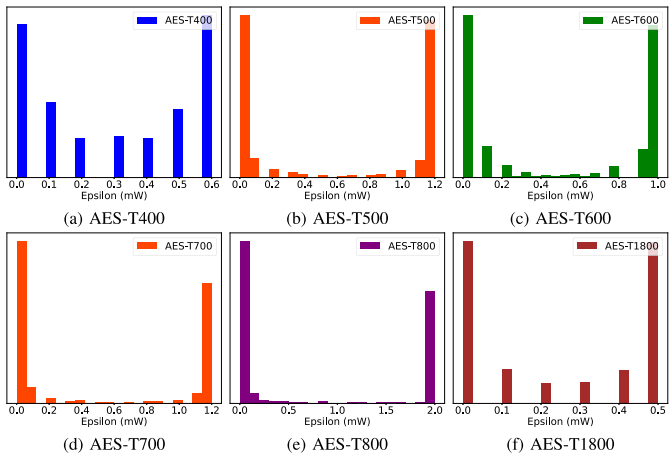
In this section, we tackle the challenge of reducing the resource utilization of HTO circuits for a more practical attack. We start by analyzing the adversarial patch value distribution. We observe that the noise power distribution is biased towards a relatively limited number of values. Using this observation, we adapted the adversarial power trace generation algorithm to have constraints on their values within a target space, thereby producing **quantized** power traces that require lower resource utilization to emulate HTOs.

TABLE I
RESOURCE UTILIZATION AND REQUIRED NOISE BUDGETS ON ASIC (T = TRANSISTORS, ϵ = REQUIRED NOISE BUDGET TO DROP HT DETECTION ACCURACY BELOW 50%)

Dataset	HTnet			ResNet-18			VGG-11			SVM		
	HT detection accuracy (%)	# T	$\epsilon(mW)$	HT detection accuracy (%)	# T	$\epsilon(mW)$	HT detection accuracy (%)	# T	$\epsilon(mW)$	HT detection accuracy (%)	# T	$\epsilon(mW)$
AES-T400	97.1	2	0.2	82.8	2	0.2	79	6	0.6	84.1	7	0.7
AES-T500	93.4	3	0.3	81.5	2	0.2	99.5	5	0.5	75	8	0.8
AES-T600	93.9	4	0.4	84.3	2	0.2	99.9	2	0.2	57	4	0.4
AES-T700	100	8	0.8	100	10	1.7	100	10	1.6	100	17	8.9
AES-T800	100	10	1.7	100	10	1.5	100	10	1.6	100	17	8.1
AES-T1800	95.7	2	0.2	75.4	1	0.1	88	2	0.2	64.7	4	0.4

TABLE II
RESOURCE UTILIZATION AND REQUIRED NOISE BUDGETS ON FPGA

Dataset	Method	HTnet		ResNet-18		VGG-11		SVM	
		# CLBs	$\epsilon(mW)$	# CLBs	$\epsilon(mW)$	# CLBs	$\epsilon(mW)$	# CLBs	$\epsilon(mW)$
AES-T400	RO	2	1	2	1	2	1	2	1
	DSP	1		1		1		1	
AES-T500	RO	2	1	2	1	2	1	2	1
	DSP	1		1		1		1	
AES-T600	RO	2	1	2	1	2	1	2	1
	DSP	1		1		1		1	
AES-T700	RO	2	1	4	2	4	2	18	9
	DSP	1		2		2		9	
AES-T800	RO	4	2	4	2	4	2	18	9
	DSP	2		2		2		9	
AES-T1800	RO	2	1	2	1	2	1	2	1
	DSP	1		1		1		1	



(a) AES-T400 (b) AES-T500 (c) AES-T600
(d) AES-T700 (e) AES-T800 (f) AES-T1800

Fig. 8. Distribution of the adversarial patch values for different target designs on HTnet model.

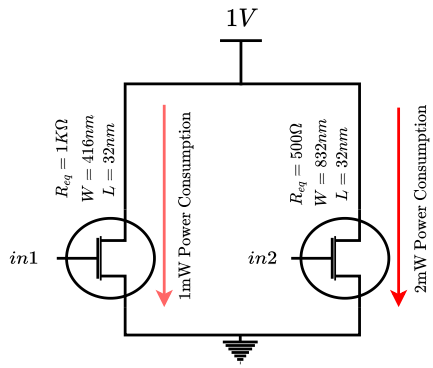


Fig. 9. Optimized ASIC adversarial noise generator.

A. Adversarial Patch Values Distribution

Due to the nature of gradient-based adversarial attacks—particularly those constrained under the L_∞ norm—adversarial samples are often biased toward the upper and lower bounds of the allowable perturbation range. In our work, following the Algorithm 1, the objective is to maximize the loss by perturbing the input within a fixed noise budget [0,ϵ]. Under the L_∞ constraint, this typically involves adjusting each input dimension independently to either 0 or $+\epsilon$, depending on the sign of the gradient. As a result, the perturbation vector often contains many components that lie exactly at the bounds of the permissible range.

Hence, our goal is to study the noise value distribution of the generated adversarial patch. Our intuition is that if the noise power distribution is biased towards a relatively limited number of values, we can exploit this to optimize the HTO circuit. Fig. 8 shows the noise power distribution of a full patch. We can clearly see that in this example, the noise power distribution is biased towards the *lower* and *upper* bound of the noise budget.

We also plotted the distribution of adversarial patches generated using the newly created datasets in Appendix Figure S2, available online, which shows that, for these additional datasets, the samples remain well constrained within the predefined upper and lower bounds of the noise budget.

B. Adversarial Patch Quantization

The previous analysis in Section V-A showed that the noise power distribution is biased towards a relatively limited number of values. Hence, a projection of the adversarial noise to a quantized space representing the most frequent values might maintain its adversarial impact.

To verify and test this hypothesis, we generate new adversarial patches with constraints on their values within a target space. The patch quantization method is exposed in Algorithm 3. The algorithm updates the adversarial patch generation process by a projection function (Line (11)), which takes a vector and rounds its values to the closest values in a subspace $\mathcal{S}$. This subspace is defined according to the most frequent values observed beforehand.

Algorithm 3: Quantized Adversarial Power Trace Generation.

Input: Data set: $\mathcal{D}$, Subspace: $\mathcal{S}$, classifier: C, noise magnitude: ε, standard deviation: σ, number of iterations: N.
Output: δ (Quantizedadversarial power trace).

Initialize $\delta \leftarrow rand()$
while $iter < N$ **do**
  for each Batch $X_i \sim \mathcal{D}$ **do**
    $\delta \leftarrow \frac{1}{|X_i|}\sum_{j=1}^{|X_i|}\{\{\alpha sign(\nabla_x J_\theta(C(x_j+\delta),\ell))\}\}$
    $\delta \leftarrow Clip_\varepsilon\{\delta\} + z \sim N(0,\sigma^2)$
    $\delta \leftarrow \mathcal{P}_\mathcal{S}(\delta)$
  end for
  $iter+ = 1$
end while

Algorithm 4: Unsynchronised Adversarial Power Trace Generation.

Input: Data set: $\mathcal{D}$, classifier: C, noise magnitude: ε, standard deviation: σ, number of iterations: N.
Output: δ Adversarial power trace.

Initialize $\delta \leftarrow rand()$
while $iter < N$ **do**
  for each Batch $X_i \sim \mathcal{D}$ **do**
    $\delta \leftarrow \frac{1}{|X_i|}\sum_{j=1}^{|X_i|}\{\{\alpha sign(\nabla_x J_\theta(C(x_j+sh(\delta)),\ell))\}\}$
    $\delta \leftarrow Clip_\varepsilon\{\delta\} + z \sim N(0,\sigma^2)$
  end for
  $iter+ = 1$
end while

function $sh\delta$ ▷//Time shift function.
  $k \leftarrow Rand(d-1)$ ▷//Select a random shift.
  Initialize $shifted\_\delta \leftarrow 0$
  $shifted\_\delta \leftarrow concat(\delta[d-k:d], \delta[0:d-k])$
  Return: $shifted\_\delta$
end Function

C. Results

Generating patches with different resolutions (0.1 mW to 1 mW) led to a significant discovery that even with the resolution of 1 mW, we reach 100% efficiency in misclassifying ML-based Trojan detectors. Therefore, to reduce the resource utilization of the adversarial power generator circuit for a more sneaky attack, we put the equivalent transistor consuming exactly the power in the noise patch. Fig. 9 depicts an optimized circuit that can consume power in the range 0 mW to 2 mW with the resolution of 1 mW with only 2 transistors.

Furthermore, we leverage different optimization processes on patches to reduce the resource utilization on ASICs and FPGAs:

- Calculating the mean value of adversarial power trace and projecting other samples to the nearest value to the upper bound, lower bound, and mean value (2 transistors)
- Mapping above the mean value to the upper bound and below ones to zero (1 transistor)

Exploiting various quantization spaces in adversarial patches decreases the number of transistors used in the design. Therefore, it minimizes the cost and area of producing the HTO in the ASIC platform. Moreover, this optimization lowers the configuration vectors, causing a significant reduction in storage units for both platforms. Our results demonstrate that we can drop the detection accuracy of the targeted model by quantizing the values of the generated patches to 2 or 1, which allows us to apply the effect of the patch using **1 transistor**.

VI. Unsynchronized Adversarial Patch

So far, we assumed that the power trace and the adversarial noise are synchronized, which can be implemented by a simple triggering signal, e.g., the HTO circuit is triggered by the first round of the AES. In this section, we relax this assumption and suppose the starting point of capturing power traces can change, and drop the synchronization assumption. We propose to generate a patch that is robust in this setting. Taking into account this characteristic of the attack, we propose a patch generator that considers the shift existing in the setup. Hence, the new problem can be formulated as follows:

$$C(x + shift(\delta, k))) \neq C(x),\ \forall\, x \sim \mu_{HT}, and$$
$$\|\delta\|_\infty \leq \varepsilon \tag{3}$$

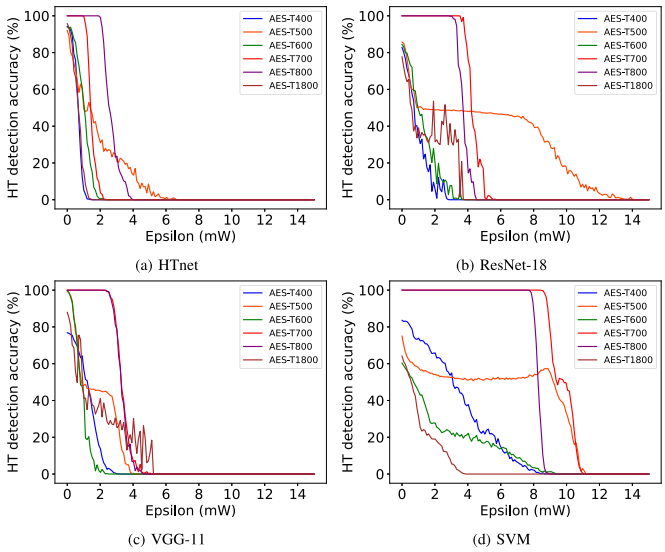
Where ε controls the magnitude of the noise vector (δ), $shift(.)$ is a function that quantifies the adversarial patch δ relatively with regard to a target power trace y given a time shift $k \in [0, d]$. Given an adversarial patch $\delta = \{\delta_j\} \forall j \in [0, d]$ that is *consumed by a circuit in a continuous loop*, the function $shift(.)$ could be expressed as:

$$shift(\delta, k) = \begin{cases} \delta(d-k+j) & \text{if} \quad j \in [0,k] \\ \delta(j-k) & \text{else} \end{cases} \tag{4}$$

To solve the problem formulated above, we propose Algorithm 4, which details the noise generation mechanism of an HTO-generated power trace that is unsynchronized with the victim circuit.

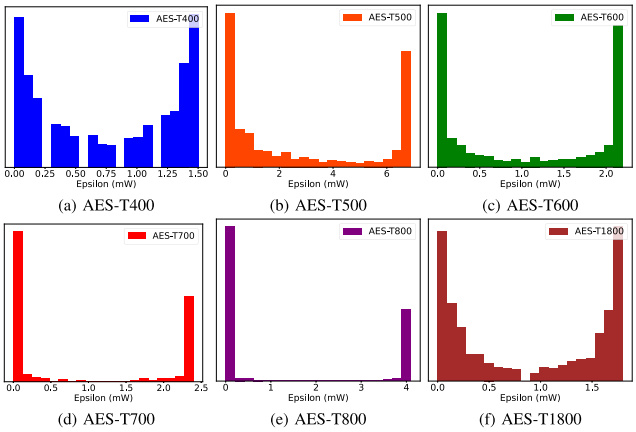
The effectiveness of different power budgets on model accuracy for the unsynchronized patches has been evaluated in Fig. 10. The figure shows that we still can fool all ML models with 100% efficiency by increasing the noise budget.

Interestingly, similar to patches generated based on Algorithm 1, as illustrated in Fig. 11, the noise power distribution is restricted to a relatively limited number of values. Hence, we leverage the same scenario discussed in Section V to optimize the circuits for unsynchronized patches. We use the same quantization methodology and reduce the values of patches to a subspace that still enables effective adversarial patches



(a) HTnet (b) ResNet-18 (c) VGG-11 (d) SVM

Fig. 10. HT detection accuracy based on changing the maximum power budget for unsynchronized patches. The stochasticity comes from the random time shift applied to the patch at inference.



(a) AES-T400 (b) AES-T500 (c) AES-T600 (d) AES-T700 (e) AES-T800 (f) AES-T1800

Fig. 11. Full unsynchronized patch distribution.

TABLE III
RESOURCE UTILIZATION AND REQUIRED NOISE BUDGETS FOR UNSYNCHRONIZED PATCHES ON ASIC

	HTnet		ResNet-18		VGG-11		SVM	
Dataset	**# T**	$\epsilon(mW)$	**# T**	$\epsilon(mW)$	**# T**	$\epsilon(mW)$	**# T**	$\epsilon(mW)$
AES-T400	1	0.7	1	0.7	1	1.1	1	1.9
AES-T500	1	1	1	0.9	1	0.8	1	1.2
AES-T600	1	1.1	1	1	1	0.8	1	0.9
AES-T700	1	1.4	1	4.2	1	3.2	1	9.3
AES-T800	1	2.5	1	3.7	1	3.2	1	8.8
AES-T1800	1	0.6	1	0.6	1	0.9	1	0.7

implementable with lower resources. The resource utilization and required noise budget to drop HT detection accuracy below 50% of the unsynchronized patches are represented in Tables III and IV

VII. COUNTERMEASURES

Our initial threat model assumes a defender deploying an ML model for HT detection without employing additional specialized countermeasures. In this section, we examine two defense strategies against HTO: one focusing on pre-processing and the other utilizing adversarial training. The pre-processing-based defense filters out irrelevant signals by targeting non-relevant frequency ranges in the input power trace. To assess this, we conduct spectral analysis to determine if the adversarially generated power trace exhibits a distinct spectral signature compared to any benign or HT-inserted circuit. Based on this analysis, we propose a pre-processing technique and discuss its limitations. Additionally, we adversarially trained the HT detector, which led to a reduction in accuracy while enabling the adaptive attack to bypass detection with a higher power budget.

TABLE IV
RESOURCE UTILIZATION AND REQUIRED NOISE BUDGETS FOR UNSYNCHRONIZED PATCHES ON FPGA

		HTnet		ResNet-18		VGG-11		SVM	
Dataset	**Method**	**# CLBs**	$\epsilon(mW)$	**# CLBs**	$\epsilon(mW)$	**# CLBs**	$\epsilon(mW)$	**# CLBs**	$\epsilon(mW)$
AES-T400	RO	2	1	2	1	4	2	4	2
	DSP	1		1		2		2	
AES-T500	RO	2	1	2	1	2	1	4	2
	DSP	1		1		1		2	
AES-T600	RO	4	2	2	1	2	1	2	1
	DSP	2		1		1		2	
AES-T700	RO	4	2	10	5	8	4	20	10
	DSP	2		5		4		10	
AES-T800	RO	6	3	8	4	8	4	18	9
	DSP	3		4		4		9	
AES-T1800	RO	2	1	2	1	2	1	2	1
	DSP	1		1		1		1	

A. Pre-Processing

The power trace can be represented in the spectral domain using the Fast Fourier Transform (FFT), which is defined by

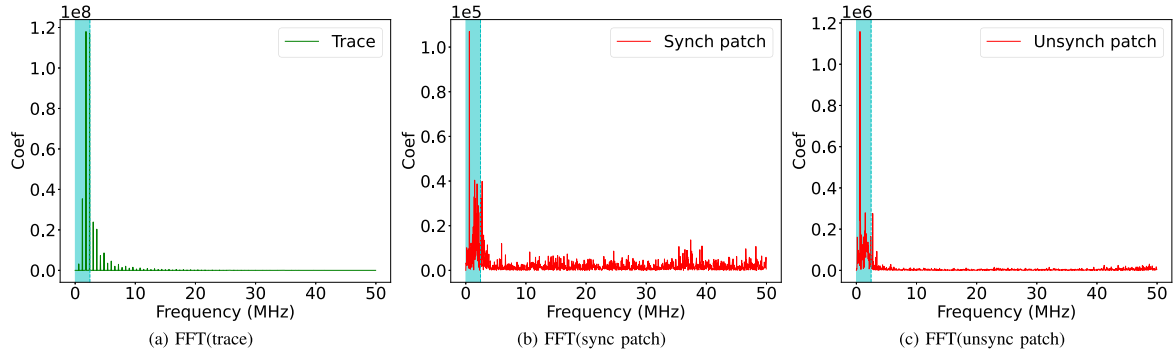
$$\mathcal{S}(k) = \sum_{n=0}^{N-1} s(n) e^{-i\frac{2\pi}{N}nk} \tag{5}$$

where N is the length of the spectral signature, $\mathcal{S}(k)$ represents the frequency domain magnitudes, and $s(n)$ is the time-domain power trace.

Our goal is to differentiate between the characteristics of adversarial power traces and those of the original circuit's power consumption in the frequency domain. Therefore, we perform an FFT on both traces and illustrate the result in Fig. 12. We observe that the adversarial noise contains frequency components beyond the expected range of the original raw signal, making it possible to apply filtering as a defense.

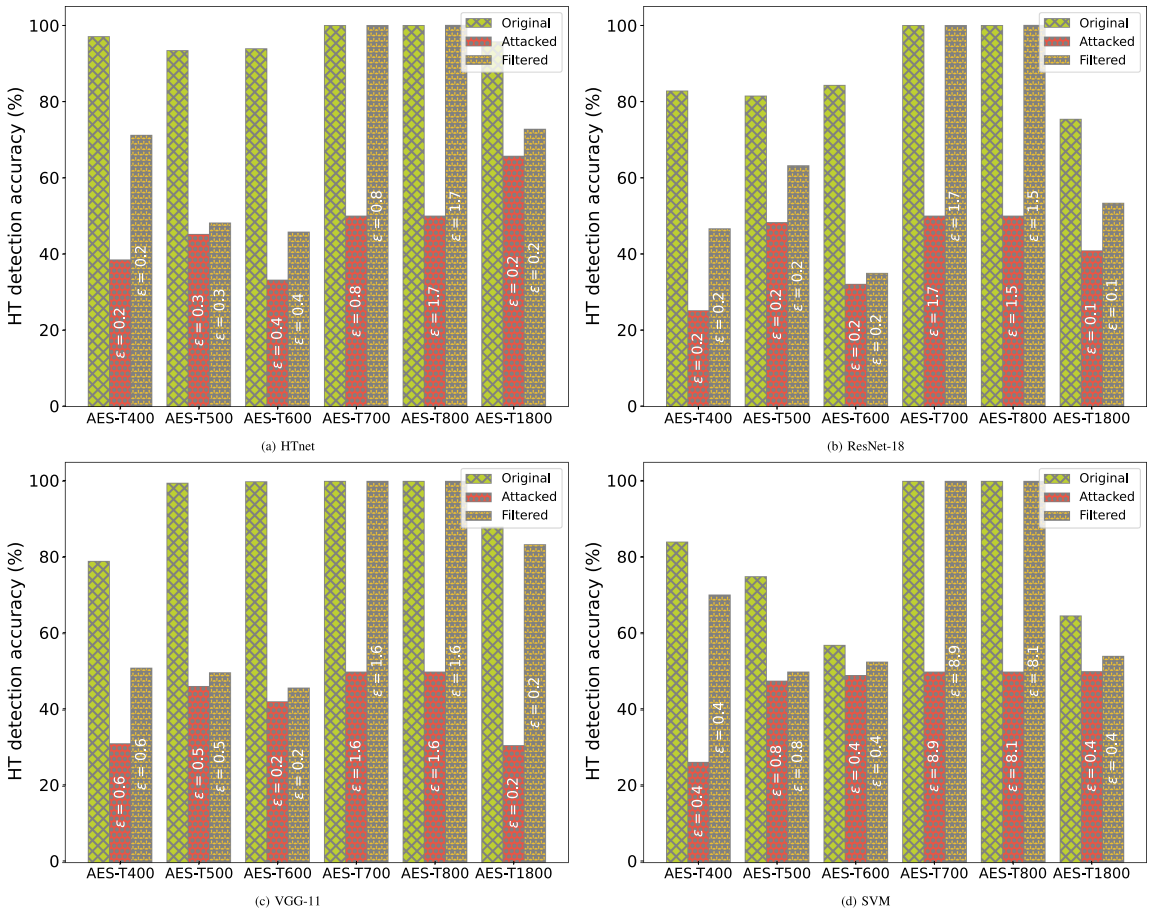
To design an appropriate filter, we consider a system that pre-processes the input signal, where the defender defines the target frequency range based on the expected power traces. We evaluate the efficiency of the HTO under this setting. We first examine the power spectral density of benign and HT-inserted circuits to identify the frequency range that represents the region of interest ($[f_{min}, f_{max}]$). A band-pass filter corresponding to this frequency range is then applied to the collected power traces before feeding it to the ML model.

Fig. 13 presents the attack success rate of HTO in the presence of the filter. The HT detection accuracy of models for $AES-T400$, $AES-T500$, $AES-T600$, and $AES-T1800$ datasets partially recovered. For the $AES-T700$ and $AES-T800$ datasets, the model's accuracy on both baseline and attacked traces remains unchanged at 100% after applying



(a) FFT(trace) (b) FFT(sync patch) (c) FFT(unsync patch)

Fig. 12. AES-T700 and the patch power distribution in frequency domain.



(a) HTnet (b) ResNet-18 (c) VGG-11 (d) SVM

Fig. 13. HT detection accuracy for original, attacked, and filtered models and required noise budgets (mW) to drop the accuracy to 50% for original models.

the filter, indicating that the attack is ineffective within the specified noise power budget. However, the attacker can still evade the HT detector with a filter by increasing the adversarial power budget as we show in Section VIII.

B. Adversarial Training

Adversarial training (AT) [22] is a state-of-the-art defense strategy against adversarial attacks. It can be formulated as follows:

$$\min_{\theta} \mathbb{E}_{(x,y)\sim\mathcal{D}} \left[\max_{\delta\in B(x,\varepsilon)} \mathcal{L}_{ce}(\theta, x+\delta, y) \right] \tag{6}$$

Where θ indicates the parameters of the classifier, $\mathcal{L}_{ce}$ is the cross-entropy loss, $(x, y) \sim \mathcal{D}$ represents the training data sampled from a distribution $\mathcal{D}$ and $B(x, \varepsilon)$ is the allowed perturbation set. The interpretation of this is that the inner maximization problem is finding the worst-case samples for the given model, and the outer minimization problem is to train a model robust to adversarial examples [22].

We used the PGD algorithm [22] to solve the inner maximization problem. We set the noise magnitude (epsilon) equal to 0 with a step size of 0.01 with a number of iterations equal to 20. We train the model for around 20 epochs to reach an acceptable classification accuracy on clean input samples.

TABLE V
ACCURACY COMPARISON BETWEEN REGULAR MODELS (R-MODELS) AND ADVERSARIALLY TRAINED MODELS (AT-MODELS)

Dataset	HTnet		ResNet-18		VGG-11		SVM	
	R-model accuracy (%)	AT-model accuracy (%)	R-model accuracy (%)	AT-model accuracy (%)	R-model accuracy (%)	AT-model accuracy (%)	R-model accuracy (%)	AT-model accuracy (%)
AES-T400	93	73.8	76.6	69.2	63.5	62.5	62.3	62.4
AES-T500	94.2	87	91.9	83.2	83.7	83	82.6	73.2
AES-T600	93.1	83	83.3	76	78.2	78	59.6	57.2
AES-T700	100	100	100	100	100	100	100	100
AES-T800	100	100	100	100	100	100	100	100
AES-T1800	97.1	93	87.9	84.1	93	93.9	59.3	58.4

TABLE VI
AT-MODELS HT DETECTION ACCURACY AND REQUIRED NOISE BUDGETS TO DROP ACCURACY BELOW 50%

Dataset	HTnet		ResNet-18		VGG-11		SVM	
	HT detection accuracy (%)	$\epsilon(mW)$	HT detection accuracy (%)	$\epsilon(mW)$	HT detection accuracy (%)	$\epsilon(mW)$	HT detection accuracy (%)	$\epsilon(mW)$
AES-T400	95	0.3	92	0.3	86.9	0.5	87	0.7
AES-T500	72.2	1.2	89	0.9	98	0.5	86.5	0.9
AES-T600	96.4	0.5	90.4	0.2	98.2	0.2	86.5	0.5
AES-T700	100	4.7	100	3.6	100	1.9	100	9.9
AES-T800	100	5.1	100	2	100	2.8	100	8.7
AES-T1800	99.3	0.3	87.1	0.4	96.7	0.2	78.5	0.4

In Tables V and VI, we present a comparative analysis of the effects of patches created using our adversarial patch technique on both AT models and undefended (regular) models. The findings show that AT reduces the efficacy of attack at lower levels of adversarial noise but becomes less effective against higher noise magnitudes (larger epsilons). This indicates that while adversarial training enhances model robustness against certain attack levels, it is less reliable against more intense adversarial inputs. Additionally, it's important to note that implementing AT, particularly at higher noise levels, as expected, incurs a decrease in models' baseline accuracy, resulting in a compromise on the overall utility of models.

VIII. ADAPTIVE ATTACK

The previous adversarial patch generation method only considered a magnitude budget in the time domain, without any spectral constraints, making the adversarial noise easily filtered. The goal of an adaptive attacker is to generate adversarial noise within a specific frequency range that aligns with the expected spectral domain of the filter. As a result, the defender would be unable to mitigate the impact of the injected noise without incurring a significant utility loss. To address this, we introduce a **spectral budget**alongside the noise magnitude budget. Specifically, we clip the noise magnitude in the time domain to fit within a noise budget and project the noise onto a subset of the frequency domain to constrain its spectral components. The problem can therefore be formulated as follows:

$$\begin{aligned} &C(x + shift(\delta, k)) \neq C(x), \forall\, x \sim \mu_{HT}, s.t. \\ &\|\delta\| = \mathcal{D}(x, x+\delta) \leq \varepsilon \\ &FFT(\delta) \in [f_{min}, f_{max}] \end{aligned} \tag{7}$$

Algorithm 5: Adaptive Adversarial Power Trace Generation.

Input: Data set: $\mathcal{D}$, classifier: C, noise magnitude: ε, standard deviation: σ, number of iterations: N.
Output: δ Adversarial power trace.

Initialize $\delta \leftarrow rand()$
while $iter < N$ **do**
  for each Batch $X_i \sim \mathcal{D}$ **do**
    $\delta \leftarrow \frac{1}{|X_i|}\sum_{j=1}^{|X_i|}\{\{\alpha sign(\nabla_x J_\theta(C(x_j + sh(\delta)), \ell))\}\}$
    $\delta \leftarrow Clip_\varepsilon\{\delta + z \sim N(0, \sigma^2)\}$
    $\delta \leftarrow Clip_{[f_{min}, f_{max}]} FFT(\delta + z \sim N(0, \sigma^2))$
  end for
  $iter += 1$
end while

Where ε controls the magnitude of the noise vector δ and $shift(.)$ is a function, expressed by (4), that quantifies the adversarial patch signal δ, relative to a target signal x, given an incidence time $k \in [0, d]$, with $k = 0$ corresponding to the synchronized case. The new constraint on δ limits the noise frequency components to an acceptable range defined by f_{min} and f_{max}, which correspond to the useful frequency range from the defender's perspective. Notice that this information is available even in the initial HT threat model.

Algorithm 5 outlines the noise generation mechanism for our adaptive attack, incorporating the frequency clipping process into the adversarial noise generation. In each iteration, after updating the noise, we project the noise back to the target spectral domain using the band-pass filter defined previously. Thus, at each iteration, we retain only the noise samples that fall within the same frequency range as the raw signal. These noise components are then aggregated into the current patch instance. The shift function is the same as in Algorithm 4. Table VII shows the required noise budget for adaptive patches to bypass the filter. The results indicate that across all datasets, the attacker must generate adversarial patches with higher power, thereby reducing the stealthiness of the patch.

IX. RELATED WORK

Several works have been studied in literature to leverage machine learning to detect HTs over the past decade [13], [23]. A novel static method for HT classification at the design phase based on SVM-based classifier and gate-level netlist features presented by Hasegawa et al. in [23]. This work was also extended to a similar framework with different ML models, including random forest (RF) [24] and neural networks [25]. A dataset of side channel signals to train a neural network for Trojan detection at run-time gathered by Faezi et al. [13]. Noor et al. [26] propose a framework for HT identification

TABLE VII
HT DETECTION ACCURACY OF NON-ADAPTIVE AND ADAPTIVE ATTACK ON FILTERED MODELS AND REQUIRED NOISE BUDGETS TO BYPASS THE FILTER BASED ON ADAPTIVE ATTACK

	HTnet			ResNet-18			VGG-11			SVM		
Dataset	Non-adaptive Attack (%)	Adaptive Attack (%)	$\epsilon(mW)$	Non-adaptive Attack (%)	Adaptive Attack (%)	$\epsilon(mW)$	Non-adaptive Attack (%)	Adaptive Attack (%)	$\epsilon(mW)$	Non-adaptive Attack (%)	Adaptive Attack (%)	$\epsilon(mW)$
AES-T400	71.2	27.8	1.3	46.7	20.3	0.4	51	22.7	1.9	70.2	26.3	1.2
AES-T500	48.2	44.8	0.4	63.2	49.4	0.4	49.8	46.6	0.6	47.6	47.6	0.9
AES-T600	45.8	35.2	0.5	35	33.2	0.3	45.8	42.3	0.3	49.1	49.1	0.5
AES-T700	100	50	4.7	100	50	15.1	100	50	5.9	100	50	15.9
AES-T800	100	50	2.8	100	50	11.9	100	50	7.0	100	50	9.2
AES-T1800	72.8	60.4	0.7	53.4	34.5	0.2	83.4	28.5	0.5	54.1	48.6	0.5

based on ML classification with features describing the dominant attributes of HTs. Features such as switching activity and net structure from the gate-level netlist have been extracted, quantified and analyzed to detect malicious activity by Zhou et al. [27]. Xiang et al. [28] constructed a random forest classifier for hardware Trojan detection by extracting the structural and signal characteristics at RTL. An experiment was done to determine the required power and delay SNR for hardware-Trojan identification by Lamech et al. [29]. Inoue et al. [30] presents a novel static method for HT classification using gate-level netlist features to identify a part of designed hardware Trojans. Iwase et al. [31] investigated a new detection technique for HTs based on frequency domain features by converting power consumption waveform data from time into the frequency domain. Estimating statistical correlation between the signals in a design and exploring how this estimation can be exploited in a clustering algorithm to identify the HT logic introduced by Cakir et al. [32]. Sarihi et al. [33] proposed a framework to insert an adversarial HT in the netlist by employing various netlist-based HT detectors in a Generative Adversarial Network (GAN)-like loop. A reinforcement learning-based approach has been proposed by Gohil et al. [34] for generating evasive hardware Trojans by selecting combinations of trigger nets that are unlikely to activate simultaneously, thereby improving stealth and evasion against pre-silicon static detection techniques. In related work, the Gohil et al. [35] introduced a novel reinforcement learning (RL)-based framework to evaluate and attack Graph Neural Networks (GNNs) applied in hardware security tasks such as intellectual property (IP) piracy detection and hardware Trojan detection.

Furthermore, ML networks can be fooled drastically by an adversarial perturbation that is not perceptible by humans [36]. Generally, the goal of the adversarial attack is to force the well-trained network to predict wrongly by generating adversarial samples. Since a majority of existing attacks focus on classification tasks, in this paper, we investigate the impact of the adversarial attack on post-silicon ML-based hardware-Trojan classifiers using power side-channel to design hardware that generates an adversarial noise to misclassify the HT-inserted circuits.

X. DISCUSSION AND CONCLUDING REMARKS

In this paper, we shed new light on the limits of using ML techniques for hardware security. Specifically, we propose a methodology for generating malicious circuitry that emulates the power consumption of an adversarial trace to deceive ML-based HT detection classifiers. We develop HTO to design circuits intended to consume a pre-generated adversarial patch (power trace), for both ASIC and FPGA platforms.

Our results indicate that achieving 100% efficiency in evading ML-based HT detection on ASIC platforms may require only a single transistor circuit. We also propose two different circuit designs—ring oscillator-based and DSP-based—that simulate sneaky adversarial power noise on FPGA platforms. Our experimental results on a Spartan 6 FPGA board demonstrate that achieving 100% efficiency in evading detection is feasible with the proposed circuit designs. Additionally, we introduce spectral-based and adversarial training countermeasures to reduce the attack's effectiveness while maintaining the model's performance. We also investigate the impact of adaptive attacks on required power budgets by simultaneously considering the magnitude of the generated patch in the time and frequency domains. Furthermore, we evaluate the effectiveness of adversarial training countermeasures in mitigating the attack's success and show that AT models suffer significant utility loss, potentially making them unsuitable for this security application.

We believe our work demonstrates a new practical method for exploiting ML vulnerabilities to adversarial noise in realistic scenarios, which is critical for hardware security. While adhering to the classical HT threat model, our approach highlights new potential threats that must be considered when using ML techniques as defenses.

Future works: Our focus was limited to circumventing detectors that rely solely on machine learning models trained on side-channel information. However, more advanced testing and validation techniques—such as side-channel analysis, reverse engineering, or physical inspection—could potentially detect the additive modifications we introduce. A comprehensive assessment of the robustness of adversarial perturbations against these broader classes of detection mechanisms is a critical next step. Such an investigation would provide a deeper understanding of the practical limitations and potential countermeasures, guiding future efforts to enhance the stealth and effectiveness of adversarial attacks.

REFERENCES

[1] M. Tehranipoor and F. Koushanfar, "A survey of hardware Trojan taxonomy and detection," *IEEE Des. test Comput.*, vol. 27, no. 1, pp. 10–25, Jan./Feb. 2010.

[2] "Hunting for backdoors in counterfeit cisco devices: F-secure press room," 2020. [Online]. Available: https://www.f-secure.com/en/press/p/hunting-for-backdoors-in-counterfeit-cisco-devices
[3] R. Jordan and R. Michael, *The Big Hack: How China Used a Tiny Chip to Infiltrate U.S. Companies*. New York, NY, USA: Bloomberg Businessweek, 2018.
[4] S. Adee, "The hunt for the kill switch," *IEEE Spectr.*, vol. 45, no. 5, pp. 34–39, May 2008.
[5] R. S. Chakraborty, S. Narasimhan, and S. Bhunia, "Hardware Trojan: Threats and emerging solutions," in *Proc. IEEE Int. High Level Des. Validation Test Workshop*, 2009, pp. 166–171.
[6] C. Bao, D. Forte, and A. Srivastava, "On reverse engineering-based hardware Trojan detection," *IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst.*, vol. 35, no. 1, pp. 49–57, Jan. 2016.
[7] B. Zhou, W. Zhang, S. Thambipillai, J. T. K. Jin, V. Chaturvedi, and T. Luo, "Cost-efficient acceleration of hardware trojan detection through fan-out cone analysis and weighted random pattern technique," *IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst.*, vol. 35, no. 5, pp. 792–805, May 2016.
[8] S. Saha, R. S. Chakraborty, S. S. Nuthakki, Anshul, and D. Mukhopadhyay, "Improved test pattern generation for hardware trojan detection using genetic algorithm and boolean satisfiability," in *Proc. 17th Int. Workshop Cryptographic Hardware Embedded Syst.*, 2015, pp. 577–596.
[9] M. Xue, C. Gu, W. Liu, S. Yu, and M. O'Neill, "Ten years of hardware trojans: A survey from the attacker's perspective," *IET Comput. Digit. Techn.*, vol. 14, no. 6, pp. 231–246, 2020.
[10] C. Dong, Y. Xu, X. Liu, F. Zhang, G. He, and Y. Chen, "Hardware trojans in chips: A survey for detection and prevention," *Sensors*, vol. 20, no. 18, 2020, Art. no. 5165.
[11] L. N. Nguyen, B. B. Yilmaz, M. Prvulovic, and A. Zajic, "A novel golden-chip-free clustering technique using backscattering side channel for hardware trojan detection," in *Proc. IEEE Int. Symp. Hardware Oriented Secur. Trust*, 2020, pp. 1–12.
[12] S. Yang, P. Chakraborty, and S. Bhunia, "Side-channel analysis for hardware trojan detection using machine learning," in *Proc. IEEE Int. Test Conf. India*, 2021, pp. 1–6.
[13] S. Faezi, R. Yasaei, and M. A. Al Faruque, "HTnet: Transfer learning for golden chip-free hardware trojan detection," in *Proc. Des., Automat. Test Europe Conf. Exhib.*, 2021, pp. 1484–1489.
[14] J. Clements and Y. Lao, "Hardware trojan attacks on neural networks," in *Proc. IEEE Int. Symp. Circuits Syst. (ISCAS)*, 2019, pp. 1–5.
[15] Z. Huang, Q. Wang, Y. Chen, and X. Jiang, "A survey on machine learning against Hardware Trojan attacks: Recent advances and challenges," *IEEE Access*, vol. 8, pp. 10796–10826, 2020.
[16] S. Faezi, R. Yasaei, A. Barua, and M. A. Al Faruque, "Brain-inspired golden chip free Hardware Trojan detection," *IEEE Trans. Inf. Forensics Secur.*, vol. 16, pp. 2697–2708, 2021.
[17] R. Yasaei, S. Faezi, and M. A. Al Faruque, "Hardware Trojan power & em side-channel dataset," *IEEE Dataport*, 2021, doi: 10.21227/9fwb-8978.
[18] A. Guesmiet al., "Defensive approximation: Securing CNNs using approximate computing," in *Proc. 26th ACM Int. Conf. Architectural Support Program. Lang. Operating Syst.*, New York, NY, USA, Association for Computing Machinery, 2021, pp. 990–1003, doi: 10.1145/3445814.3446747.
[19] I. J. Goodfellow, J. Shlens, and C. Szegedy, "Explaining and harnessingadversarial examples," 2014.
[20] N. Carlini and D. Wagner, "Towards evaluating the robustness of neural networks," in *Proc. IEEE Symp. Secur. Privacy*, 2017, pp. 39–57.
[21] B. Shakya, T. He, H. Salmani, D. Forte, S. Bhunia, and M. Tehranipoor, "Benchmarking of Hardware Trojans and maliciously affected circuits," *J. Hardware Syst. Secur.*, vol. 1, no. 1, pp. 85–102, 2017.
[22] A. Madry, A. Makelov, L. Schmidt, D. Tsipras, and A. Vladu, "Towards-deep learning models resistant to adversarial attacks," 2019.
[23] K. Hasegawa, M. Oya, M. Yanagisawa, and N. Togawa, "Hardware Trojans classification for gate-level netlists based on machine learning," in *Proc. IEEE 22nd Int. Symp. On-Line Testing Robust System Des.*, 2016, pp. 203–206.
[24] K. Hasegawa, M. Yanagisawa, and N. Togawa, "Trojan-feature extraction at gate-level netlists and its application to Hardware-Trojan detection using random forest classifier," in *Proc. IEEE Int. Symp. Circuits Syst.*, 2017, pp. 1–4.
[25] K. Hasegawa, M. Yanagisawa, and N. Togawa, "Hardware Trojans classification for gate-level netlists using multi-layer neural networks," in *Proc. IEEE 23rd Int. Symp. On-Line Testing Robust System Des.*, 2017, pp. 227–232.
[26] N. Q. M. Noor, N. N. A. Sjarif, N. H. F. M. Azmi, S. M. Daud, and K. Kamardin, "Hardware Trojan identification using machine learning-based classification," *J. Telecommunication, Electron. Comput. Eng.*, vol. 9, no. 3–4, pp. 23–27, 2017.
[27] E.-R. Zhou, S.-Q. Li, J.-H. Chen, L. Ni, Z.-X. Zhao, and J. Li, "A novel detection method for Hardware Trojan in third party IP cores," in *Proc. Int. Conf. Inf. System Artif. Intell.*, 2016, pp. 528–532.
[28] Y. Xiang, L. Li, and W. Zhou, "Random forest classifier for Hardware Trojan detection," in *Proc. 12th Int. Symp. Comput. Intell. Des.*, 2019, vol. 1, pp. 134–137.
[29] C. Lamech, R. M. Rad, M. Tehranipoor, and J. Plusquellic, "An experimental analysis of power and delay signal-to-noise requirements for detecting Trojans and methods for achieving the required detection sensitivities," *IEEE Trans. Inf. Forensics Secur.*, vol. 6, no. 3, pp. 1170–1179, Sep. 2011.
[30] T. Inoue, K. Hasegawa, M. Yanagisawa, and N. Togawa, "Designing Hardware Trojans and their detection based on a SVM-based approach," in *Proc. IEEE 12th Int. Conf. ASIC*, 2017, pp. 811–814.
[31] T. Iwase, Y. Nozaki, M. Yoshikawa, and T. Kumaki, "Detection technique for Hardware Trojans using machine learning in frequency domain," in *Proc. IEEE 4th Glob. Conf. Consum. Electron.*, 2015, pp. 185–186.
[32] B. Cakır and S. Malik, "Hardware Trojan detection for gate-level ICs using signal correlation based clustering," in *Proc. Des., Automat. Test Europe Conf. Exhib.*, 2015, pp. 471–476.
[33] A. Sarihi, P. Jamieson, A. Patooghy, and A.-H. A. Badawy, "TrojanForge: Generating adversarial Hardware Trojan examples using reinforcement learning," in *Proc. ACM/IEEE Int. Symp. Mach. Learn. CAD*, 2024, pp. 1–7.
[34] V. Gohil, H. Guo, S. Patnaik, and J. Rajendran, "Attrition: Attacking static Hardware Trojan detection techniques using reinforcement learning," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2022, pp. 1275–1289.
[35] V. Gohil, S. Patnaik, D. Kalathil, and J. Rajendran, "{ AttackGNN }:{ Red-Teaming }{ GNNs} in hardware security using reinforcement learning," in *Proc. 33rd USENIX Secur. Symp.*, 2024, pp. 73–90.
[36] C. Szegedyet al., "Intriguing properties of neural networks," in *Proc. 2nd Int. Conf. Learn. Representations (ICLR)*, 2014. [Online]. Available: http://arxiv.org/abs/1312.6199

Behnam Omidi received the MS degree in computer engineering from the Amirkabir University of Technology, Iran. He is currently working toward the PhD degree with the Department of Electrical and Computer Engineering, George Mason University, Fairfax, VA, USA. His research interests include detecting malicious activity on hardware, micro-architectural side-channel attacks, and designing circuits for supporting hardware security.

Ihsen Alounai (Senior Member, IEEE) received the PhD degree from Université Polytechnique Hauts-deFrance, Valenciennes, France. He is currently a senior lecturer with the Centre for Secure Information Technologies, Queen's University Belfast, Belfast, U.K. His research interests include machine learning security and privacy, neuromorphic computing, and systems reliability and security.

Khaled N. Khasawneh (Member, IEEE) received the PhD degree in computer science from the University of California, Riverside. He is currently an assistant professor with Electrical and Computer Engineering Department, George Mason University, Fairfax, VA, USA. His research focuses on architecture support for security.